



北京大學
PEKING UNIVERSITY

Hierarchical Memory

2026/05/13



Nested Learning: The Illusion of Deep Learning Architecture

G o o g l e

Nested Learning: The Illusion of Deep Learning Architecture

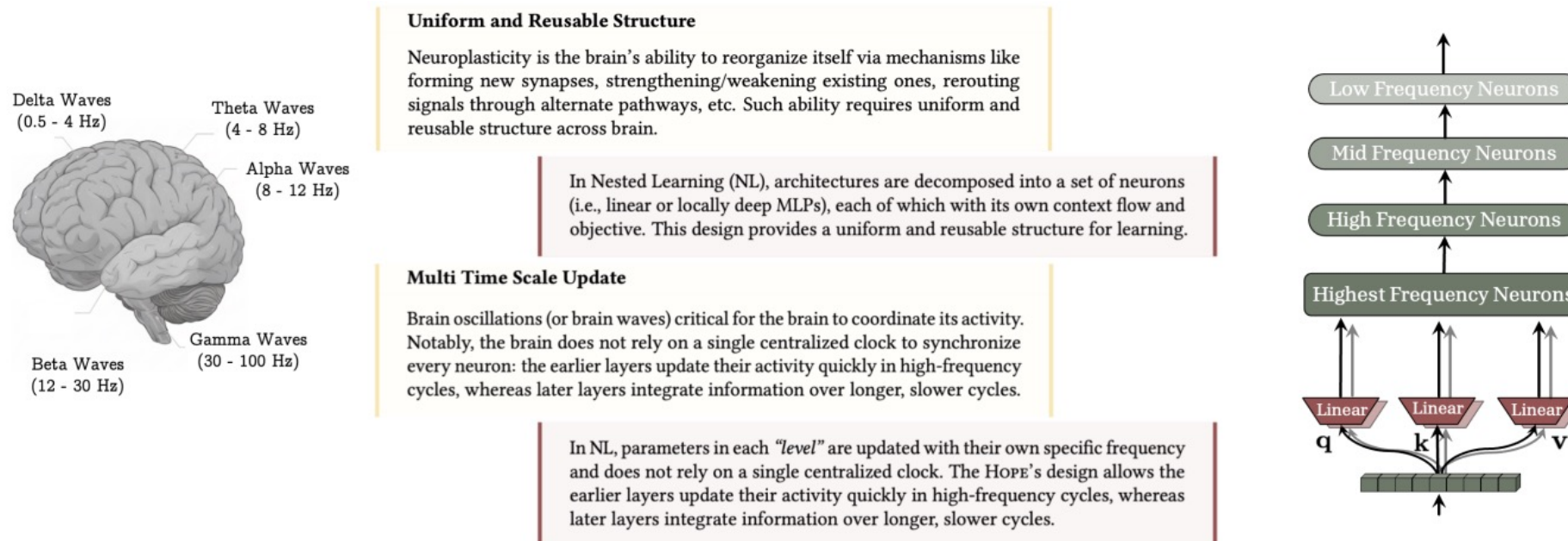


Figure 1: The uniform and reusable structure as well as multi time scale update in the brain are the key components to unlock the continual learning in humans. Nested Learning (NL) allows for multi time-scale update for each component of the brain, while showing that well-known architectures such as Transformers are in fact linear layers with different frequency updates.

- 当前模型的知识来源于两者：上下文包含的短期记忆，内化于MLP的长期记忆
- 模仿人脑的记忆巩固机制和多时间尺度更新提出了嵌套学习范式

Associative Memory

Definition 1 (Associative Memory). Given a set of keys $\mathcal{K} \subseteq \mathbb{R}^{d_k}$ and values $\mathcal{V} \subseteq \mathbb{R}^{d_v}$, associative memory is an operator $\mathcal{M}(\cdot)$ that maps the set of keys $\mathcal{K}$ to values $\mathcal{V}$. To learn such mapping from the data, an objective $\tilde{\mathcal{L}}(\cdot; \cdot)$ measures the quality of the mapping and $\mathcal{M}$ can be computed by:

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \tilde{\mathcal{L}}(\mathcal{M}(\mathcal{K}); \mathcal{V}). \quad (6)$$

- key/value 不局限于 token，也可以是梯度、子序列等任意上下文事件。
- 算子 $\mathcal{M}$ 本身是记忆，学习 $\mathcal{M}$ 的过程就是从数据中获得有效记忆。
- 模型训练可以理解为：将 key-value 映射关系压缩进模型参数中。
- 神经网络整体可视为多层级、嵌套、可能并行的记忆系统。

Associative Memory

A Simple Example of MLP Training. We start with a simple example, in which we aim to train a 1-layer MLP (parameterized with W) for task $\mathcal{T}$ and on dataset $\mathcal{D}_{\text{train}} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{D}_{\text{train}}|}\}$ by optimizing the objective $\mathcal{L}(\cdot; \cdot)$ with gradient descent. In this case, the training process objective is to solve the following optimization problem:

$$W^* = \arg \min_W \mathcal{L}(W; \mathcal{D}_{\text{train}}), \quad (7)$$

whose optimization by (stochastic/online) gradient descent results in a weight update rule:

$$W_{t+1} = W_t - \underbrace{\eta_{t+1} \nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1})}_{\text{Surprise}} = W_t - \underbrace{\eta_{t+1} \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})}_{\text{Surprise in the Output}} \otimes \mathbf{x}_{t+1}, \quad \text{where } \mathbf{x}_{t+1} \sim \mathcal{D}_{\text{train}}, \quad (8)$$

“局部意外信号” (Local Surprise Signal, LSS)

$$u_{t+1} = \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$$

$$W_{t+1} = \arg \min_W \langle W \mathbf{x}_{t+1}, u_{t+1} \rangle + \frac{1}{2\eta_{t+1}} \|W - W_t\|_2^2 = \arg \min_W \langle W \mathbf{x}_t, \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \rangle + \frac{1}{2\eta_{t+1}} \|W - W_t\|_2^2.$$

- 训练过程就是一个学习记忆 $M(\cdot) = W(\cdot)$ 的过程, 这个记忆系统学习将输入数据点 $\mathbf{x}_{t+1}$ 映射到它们对应的 u_{t+1}
- 这是一个单层级 (1-level) 的学习系统

Associative Memory

momentum-based variant

$$W_{t+1} = W_t - \mathbf{m}_{t+1},$$

$$\mathbf{m}_{t+1} = \mathbf{m}_t + \eta_{t+1} \nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1}) = \mathbf{m}_t + \eta_{t+1} \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \otimes \mathbf{x}_{t+1}.$$

$$m_{t+1} = \arg \min_m -\langle m, u_{t+1} \rangle + \frac{1}{2(1-\beta)} \|m - m_t\|_F^2$$

带动量的梯度下降是一个**双层级 (2-level)** 的缔合记忆系统

- 内层 (Inner Level) : 动量项本身是一个key-less的缔合记忆。它通过优化目标函数, 将历史梯度 $u_t, u_{t-1}, \dots$ 压缩到其参数 m 中。这是一个快速更新的记忆。
- 外层 (Outer Level) : 权重 W_t 是慢速更新的记忆。它利用内层记忆 m_{t+1} 的输出来进行更新。
- 将优化器本身也看作是一个学习模块
- Fast Weight Programmers, FWPs

Associative Memory

An Example of Architectural Decomposition. In the next example, we replace the MLP module in our previous example with a linear attention (Katharopoulos et al. 2020). That is, we aim to train a 1-layer linear attention for task $\mathcal{T}$ and on a sequence of $\mathcal{D}_{\text{train}} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{D}_{\text{train}}|}\}$ by optimizing the objective $\mathcal{L}$ with gradient descent. Recalling the unnormalized linear attention formulation:

$$\mathbf{k}_t = W_k \mathbf{x}_t, \quad \mathbf{v}_t = W_v \mathbf{x}_t, \quad \mathbf{q}_t = W_q \mathbf{x}_t, \quad (14)$$

$$\mathcal{M}_t = \mathcal{M}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad (15)$$

$$y_t = \mathcal{M}_t \mathbf{q}_t. \quad (16)$$

$$\mathcal{M}_{t+1} = \arg \min_{\mathcal{M}} -\langle \mathcal{M} \mathbf{k}_{t+1}, \mathbf{v}_{t+1} \rangle + \frac{1}{2} \|\mathcal{M} - \mathcal{M}_t\|_2^2$$

$$\Rightarrow \mathcal{M}_{t+1} = \mathcal{M}_t - \nabla \tilde{\mathcal{L}}(\mathcal{M}_t; \mathbf{k}_{t+1}, \mathbf{v}_{t+1}) = \mathcal{M}_t + \mathbf{v}_{t+1} \mathbf{k}_{t+1}^\top,$$

也可看作是一个**双层级 (2-level)** 的缔合记忆系统

- 内层 (Inner Level) : 注意力记忆 $\mathcal{M}_t$ 的更新。这是一个快速过程, 在每个时间步都发生, 旨在将当前的键值对 $\langle \mathbf{k}_t, \mathbf{v}_t \rangle$ 压缩进记忆状态 $\mathcal{M}_t$ 。
- 外层 (Outer Level) : 投影矩阵 W_q, W_k, W_v 的训练。
- 两个层级的优化梯度流是互相独立的

Nested Learning: The Illusion of Deep Learning Architecture

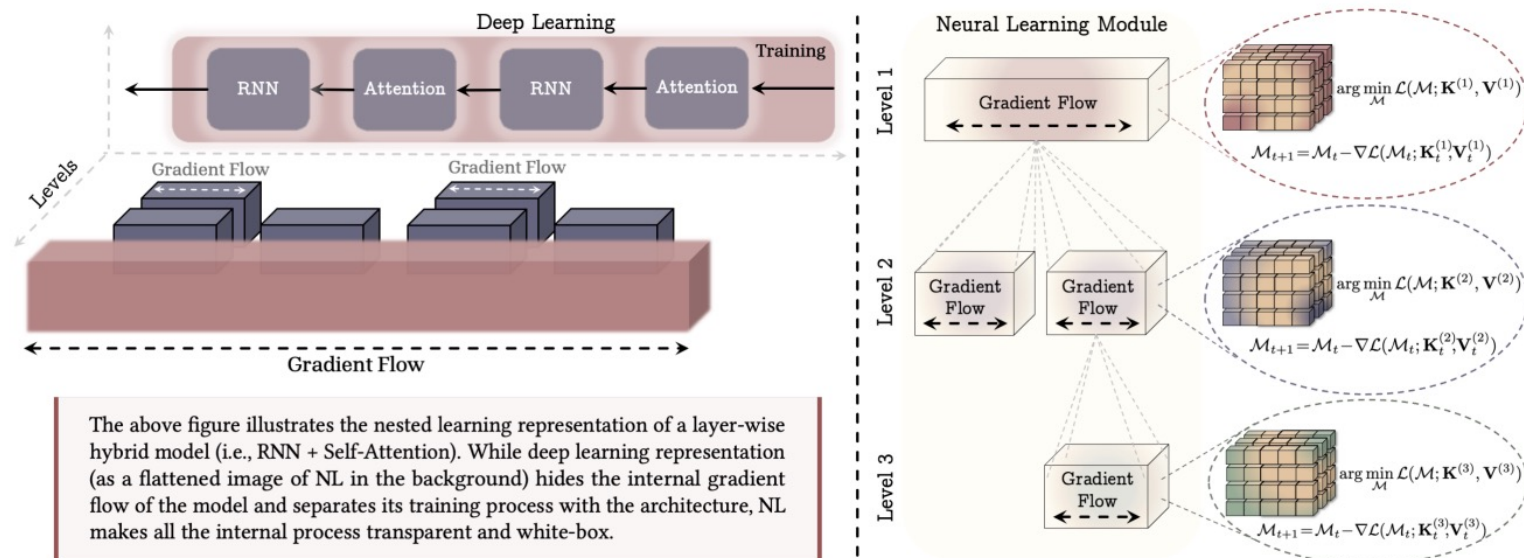


Figure 2: Nested Learning Paradigm that represent a machine learning model and its training procedure as a set of nested optimization problems. **(Left)** An example of Hybrid architecture. While deep learning perspective, as the flattened image of NL, does not provide insight about the depth of computation in the blocks, NL transparently represent all the inner gradient flows. **(Right)** A Neural Learning Module: A computational model that learns how to compress its own context flow. For example, the first level corresponds to the model's most outer-loop training, often refer to as "pre-training" step.

$$\begin{aligned}
 & \{(\mathcal{L}_i^{(k)}, C_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k} \\
 & C_i = \{(k_j^{(i)}, v_j^{(i)})\}_{j=1}^{L_i}
 \end{aligned}$$

$$\theta_{i,t+1}^{(k)} = \arg \min_{\Phi_i^{(k)}} \langle \Phi_i^{(k)} k_{t+1}^{(i)}, -\nabla \mathcal{L}_i^{(k)}(\theta_{i,t}^{(k)}; k_{t+1}^{(i)}, v_{t+1}^{(i)}) \rangle + \frac{1}{2\eta_{i,t+1}^{(k)}} \|\Phi_i^{(k)} - \theta_{i,t}^{(k)}\|_2^2,$$

$$\text{where } (k_{t+1}^{(i)}, v_{t+1}^{(i)}) \sim C_i^{(k)} \text{ and } \Phi_i^{(k)} \in \Theta_i^{(k)}.$$

利用梯度更新频率将这个嵌套架构分层级

- 对任意组件A, 将其更新频率 f_A 定义为单位时间内的更新次数
- 我们称 A 比 B 更快, 记作 $A > B$, 当满足以下任一条件:

$$f_A > f_B \text{ 或 } f_A = f_B, \text{ 且 } B_t \text{ 依赖于 } A_t$$

Nested Learning: The Illusion of Deep Learning Architecture

将层级之间信息传递分为几种主要类型：

- Direct Connection of Levels $\mathcal{M}^{(0)}(\cdot) := \mathcal{M}^{(0)}(\cdot; \mathcal{M}^{(1)}(\cdot))$ $\mathcal{M}^{(0)}(\cdot) := \mathcal{M}^{(0)}(\cdot; \mathcal{M}^{(1)}(\cdot; \mathcal{C}^{(1)}))$
- Knowledge Transfer via Backpropagation $\begin{cases} \Theta_{t+1}^{(1)} = \Theta_t^{(1)} - \eta_{t+1}^{(1)} \delta_1 \hat{\mathbf{x}}_{t+1}^\top, \\ \Theta_{t+1}^{(0)} = \Theta_t^{(0)} - \eta_{t+1}^{(0)} \delta_0 \mathbf{x}_{t+1}^\top, \end{cases}$
- Knowledge Transfer via Initialization $\Theta_0^{(1)} = \arg \min_{\Phi} \mathbb{E}_{C \sim \mathcal{C}^{(0)}} [\ell(\mathcal{M}^{(1)}(\cdot; \Phi), C)]$
- Connections with Generation $\begin{cases} \Theta^{(1)} = g(\mathcal{M}^{(0)}; \mathcal{L}^{(0)}; \mathcal{C}^{(0)}; \Theta^{(0)}), \text{ or} \\ \Theta^{(0)} = g(\mathcal{M}^{(1)}; \mathcal{L}^{(1)}; \mathcal{C}^{(1)}; \Theta^{(1)}), \end{cases}$ (Weight Generation) $\begin{cases} \mathcal{C}^{(1)} = g(\mathcal{M}^{(0)}; \mathcal{L}^{(0)}; \mathcal{C}^{(0)}; \Theta^{(0)}), \text{ or} \\ \mathcal{C}^{(0)} = g(\mathcal{M}^{(1)}; \mathcal{L}^{(1)}; \mathcal{C}^{(1)}; \Theta^{(1)}). \end{cases}$ (Context Generation)

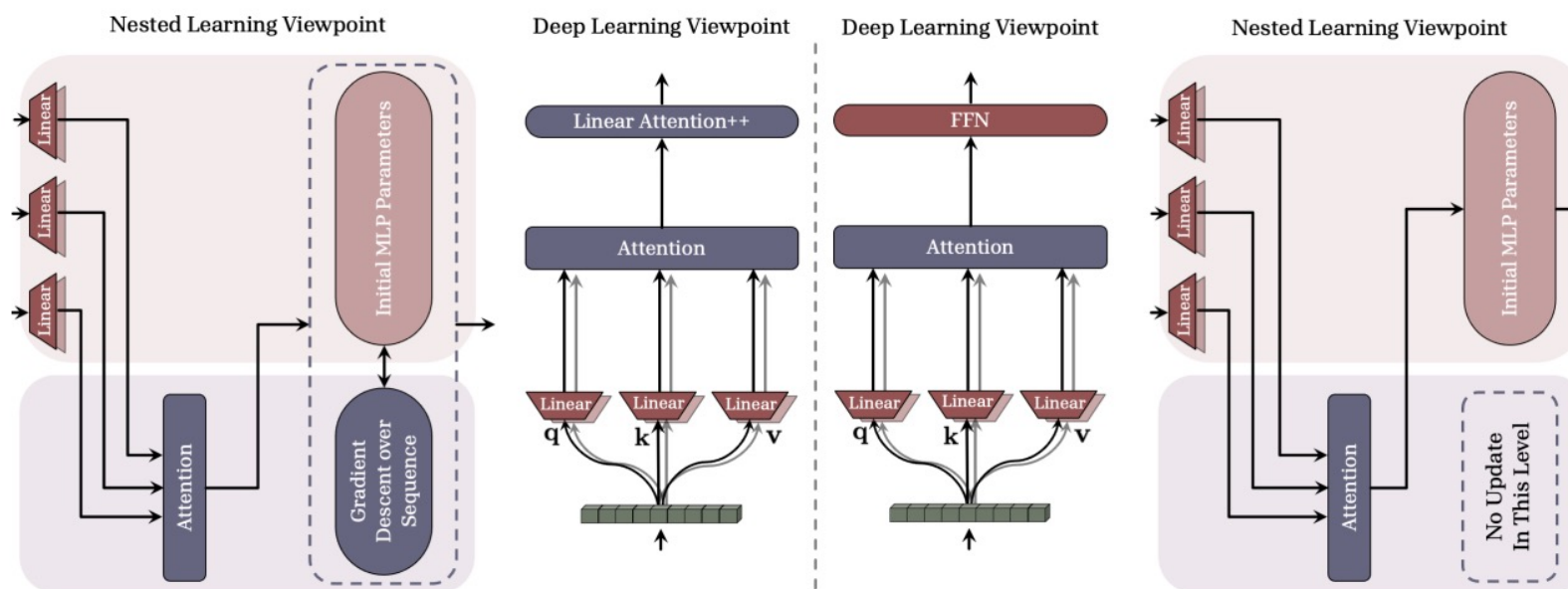


Figure 3: An example of comparing a FFN (e.g., MLP) with linear attention in a Transformer-based backbone, optimizing with gradient descent. The red components are blocks in the first level (with frequency 1), while blue components are blocks in the second level (frequency L). Linear attention with learnable initial memory state (referred to as Linear Attention++) is the same as an MLP layer but with in-context learning ability and adaptation to the input sequence.

连续谱记忆系统 (Continuum Memory System, CMS)

Given the definition of frequency (Definition 2), Continuum Memory System (CMS) is formalized as a chain of MLP blocks $\text{MLP}^{(f_1)}(\cdot), \dots, \text{MLP}^{(f_k)}(\cdot)$, each of which associated with a chunk size of $C^{(\ell)} := \frac{\max_i C^{(i)}}{f_t}$ such that given input $x = \{\mathbf{x}_1, \dots, \mathbf{x}_T\}$ the output of the chain is calculated as (we disregard normalizations for the sake of clarity):

$$\mathbf{y}_t = \text{MLP}^{(f_k)}(\text{MLP}^{(f_{k-1})}(\dots \text{MLP}^{(f_1)}(\mathbf{x}_t))), \quad (70)$$

where the parameters of ℓ -th MLP block, i.e., $\theta^{(f_\ell)}$, are updated every $C^{(\ell)}$ steps:

$$\theta_{i+1}^{(f_\ell)} = \theta_i^{(f_\ell)} - \begin{cases} \sum_{t=i-C^{(\ell)}}^i \eta_t^{(\ell)} f(\theta_t^{(f_\ell)}; \mathbf{x}_t) & \text{if } i \equiv 0 \pmod{C^{(\ell)}}, \\ 0 & \text{otherwise.} \end{cases} \quad (71)$$

- **高频模块** 负责处理和压缩短期的、局部的上下文信息。
- **低频模块** 负责整合和存储长期的、抽象的知识。
- Transformer 视作 $k=1$ 的特例，FFN为更新频率趋近0的低频模块，注意力层视作无状态的高频模块

Nested Learning: The Illusion of Deep Learning Architecture

Hope: A Self-Referential Learning Module with Continuum Memory

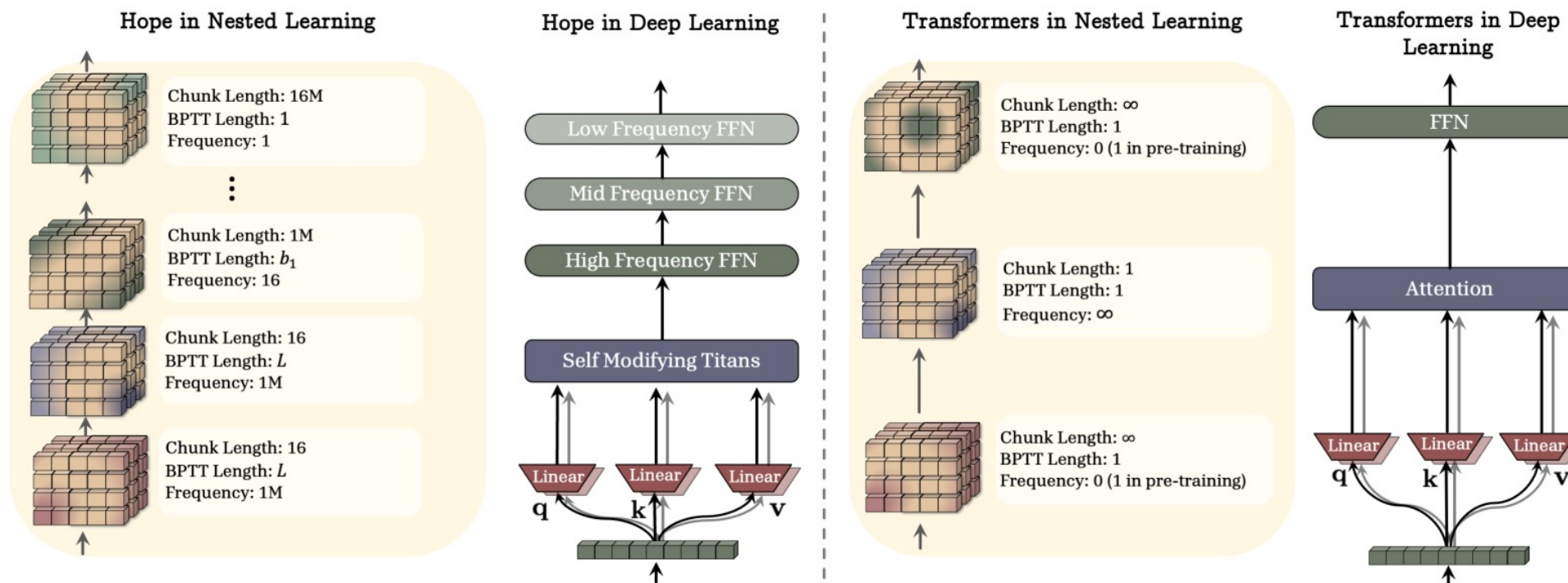


Figure 5: A comparison of Hope architectural backbone with Transformers (Normalization and potential data-dependent components are removed for the sake of clarity).

- Multi-Frequency FFNs
- Self Modifying Titans for token-level dynamics

Nested Learning: The Illusion of Deep Learning Architecture

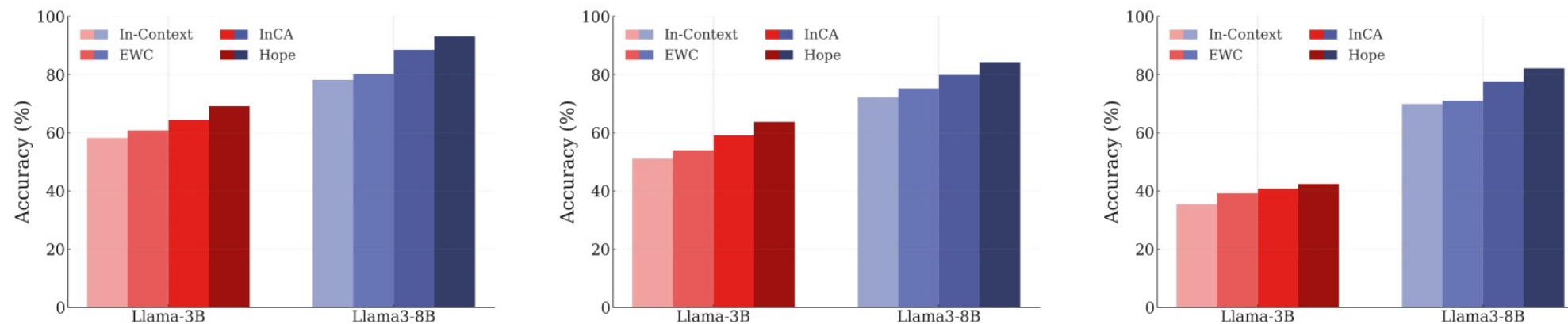


Figure 6: Class-incremental learning in the text classification domain on (Left) CLINC dataset (Larson et al. 2019), (Middle) Banking dataset (Casanueva et al. 2020), and (Right) DBpedia dataset (Auer et al. 2007). HOPE-enhanced architecture achieves the best accuracy among other continual learning methods, including ICL.

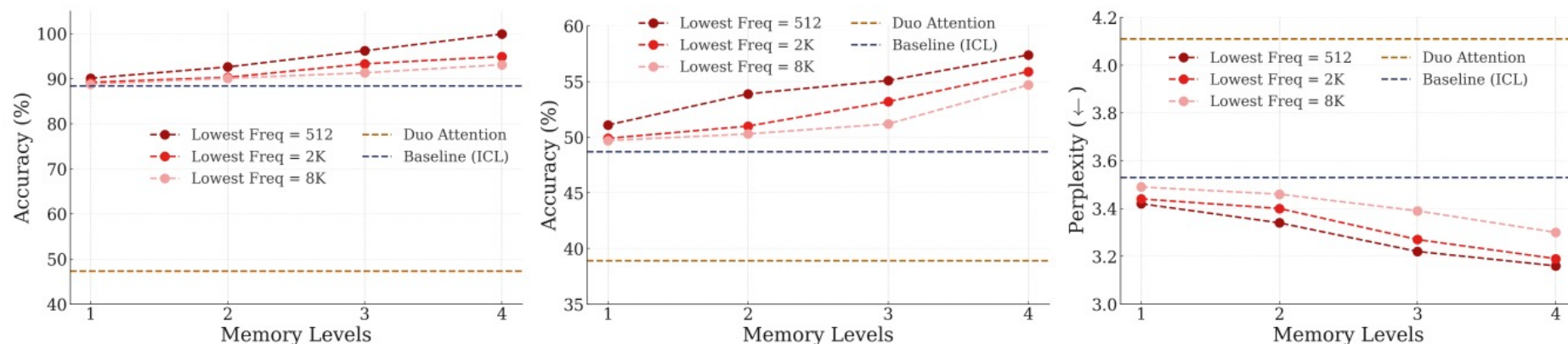


Figure 7: The effect of memory levels on the in-context learning performance of the model in (Left) MK-NIAH of RULER (Hsieh et al. 2024), (Middle) LongHealth (Adams et al. 2025), (Right) QASPER (Dasigi et al. 2021) benchmarks. Note that for QASPER benchmark (Right), the lower values indicate better performance.

Nested Learning: The Illusion of Deep Learning Architecture

Table 1: Needle-In-A-Haystack experiments with: (1) Single needle with three levels of difficulty: single-needle tasks—S-NIAH-1 (passkey retrieval), S-NIAH-2 (numerical needle), and S-NIAH-3 (UUID-based needle); (2) multi-query; (3) multi-key; and (4) multi-value settings of the benchmark.

Model	S-NIAH-1 (pass-key retrieval)			S-NIAH-2 (number in haystack)			S-NIAH-3 (uuid in haystack)		
	4K	8K	16K	4K	8K	16K	4K	8K	16K
Transformer	88.6	76.4	79.8	100	98.8	94.2	78.0	69.2	40.8
HOPE-Attention	100	100	100	100	98.4	94.4	76.8	68.8	42.4
RWKV-7	100	100	99.6	93.8	44.8	12.6	63.8	13.2	5.8
Comba	100	100	99.4	92.6	47.2	13.4	62.4	13.8	7.4
DLA	96.4	71.2	44.0	79.6	42.6	28.2	18.2	8.8	4.0
Titans	100	100	100	99.6	84.6	75.4	74.2	42.8	21.2
HOPE	100	100	100	99.2	88.4	78.2	73.2	46.2	24.8

Model	MK-NIAH-1 (multi-key line retrieval)			MQ-NIAH (multi-query)			MV-NIAH (multi-value)		
	4K	8K	16K	4K	8K	16K	4K	8K	16K
Transformer	79.4	83.0	61.4	58.9	48.0	29.8	37.5	34.1	21.5
HOPE-Attention	80.2	84.8	60.8	60.4	47.8	30.6	35.2	34.4	24.8
RWKV-7	21.4	18.8	9.6	20.4	14.8	8.6	16.2	13.4	6.8
Comba	21.4	19.4	8.2	21.8	15.2	6.4	16.5	13.5	7.2
DLA	27.4	20.0	11.8	26.4	22.0	6.4	25.6	12.8	9.6
Titans	26.4	23.6	8.2	22.8	19.8	9.4	24.6	15.1	8.2
HOPE	29.4	24.8	14.8	31.7	24.8	14.2	31.4	17.2	11.4

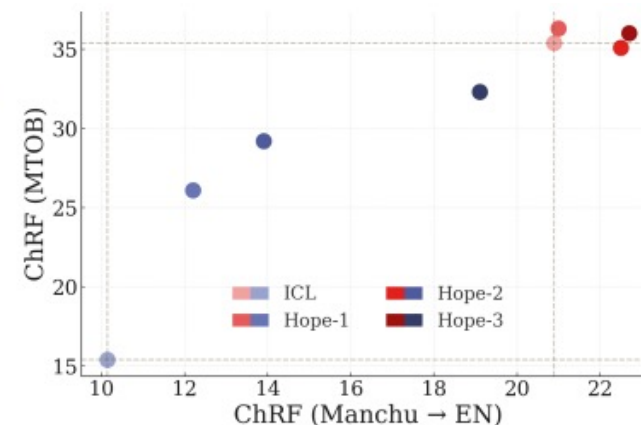


Figure 8: Continual Translation of a Novel Language (CTNL) task. Red points are the results with only one language. Blue points are the results in the continual learning setup.

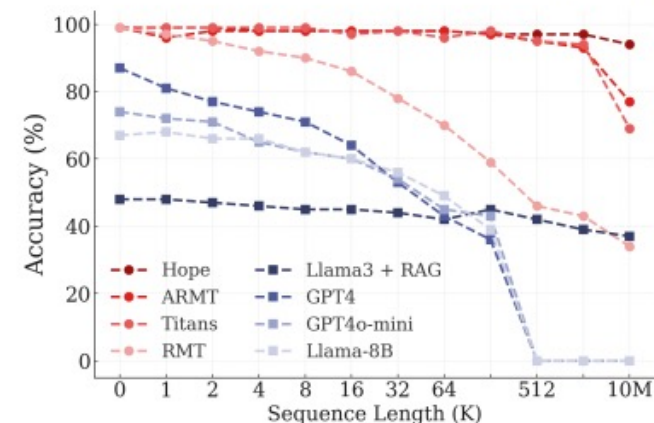


Figure 9: BABILong Benchmark. Red points are the results of fine-tuned models, and Blue points are the large models' zero-shot results.

Nested Learning: The Illusion of Deep Learning Architecture

Table 2: Performance of models on language modeling and common-sense reasoning tasks.

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	SIQA acc ↑	BoolQ acc ↑	Avg. ↑
760M params / 30B tokens											
Transformer++	24.18	24.27	37.1	67.2	43.8	53.0	65.6	33.4	39.1	61.7	50.11
Samba*	21.07	22.85	39.2	68.9	47.8	53.1	65.8	34.9	38.9	63.1	51.46
RetNet	25.77	24.19	34.5	66.8	41.2	51.9	63.6	32.5	38.8	56.2	48.19
DeltaNet	24.52	24.38	36.8	67.3	44.5	51.8	64.2	32.7	39.6	60.1	49.63
RWKV-7	23.75	23.08	37.1	67.3	47.6	52.2	64.7	34.2	39.4	61.9	50.55
Comba	22.41	22.19	37.5	66.9	48.2	52.4	65.1	34.1	40.1	62.8	50.89
TTT	24.17	23.51	34.7	67.3	43.9	51.0	64.5	33.8	40.2	59.6	47.32
Miras (Memora)	22.28	22.31	38.2	67.8	49.3	53.3	63.6	36.1	40.9	63.0	51.53
DLA	23.12	22.09	36.1	68.0	47.9	52.7	65.8	34.6	39.1	59.6	50.48
Titans	20.08	21.52	38.1	69.1	48.5	52.7	66.2	35.7	40.3	62.8	51.68
HOPE	18.68	20.07	38.8	69.2	49.1	53.6	66.8	36.1	41.2	63.4	52.28
1.3B params / 100B tokens											
Transformer++	17.92	17.73	42.6	71.4	52.3	54.1	69.9	36.5	41.8	58.4	53.38
Samba*	16.15	13.21	45.2	71.5	53.8	55.8	69.1	36.7	40.6	63.0	54.46
RetNet	18.91	17.04	41.2	71.3	49.1	55.2	67.5	34.1	41.4	61.0	52.60
DeltaNet	18.62	17.10	41.6	70.1	49.4	52.7	67.6	35.2	39.7	54.8	51.39
RWKV-7	18.44	15.96	46.7	72.4	54.9	57.5	71.6	38.2	40.7	60.4	55.30
Comba	18.16	14.87	46.9	73.1	54.5	57.7	72.0	39.1	40.2	60.6	55.39
TTT	18.42	14.51	46.8	72.9	55.2	59.0	71.8	39.5	39.8	59.6	55.58
Miras (Memora)	15.90	12.04	48.7	73.1	56.0	57.4	71.5	37.9	40.2	61.3	55.76
DLA	16.31	12.29	44.5	70.6	53.9	54.2	69.6	36.0	40.8	60.2	53.72
Titans	15.60	11.41	49.1	73.1	56.3	59.8	72.4	40.8	42.1	61.0	56.82
HOPE	14.39	10.08	51.0	73.9	57.5	61.2	73.8	42.7	42.8	61.4	58.04

* is a hybrid of attention + linear RNN (Ren et al. 2024).

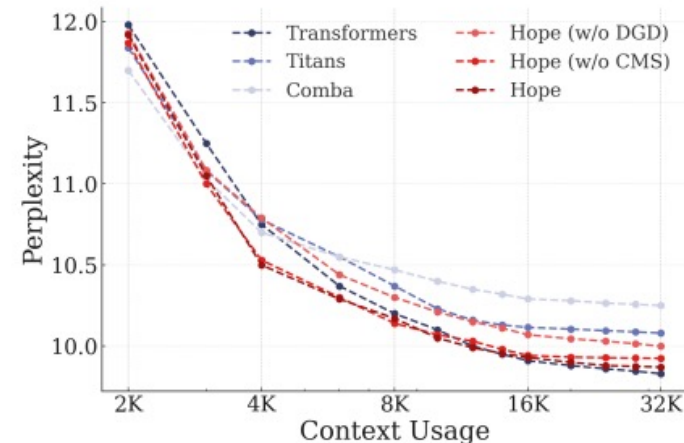


Figure 10: The effect of context usage on the perplexity of the model. We expect the perplexity of a model with powerful memory management to decrease with more context.



Pretraining with hierarchical memories: separating long-tail and common knowledge

A p p l e I C L R 2 0 2 6

Pretraining with hierarchical memories

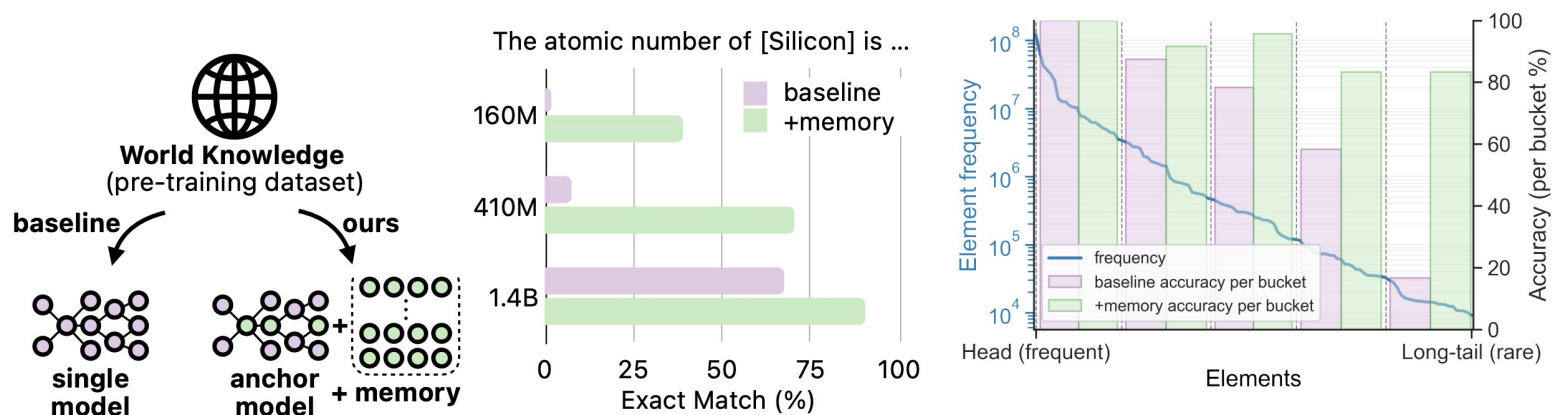


Figure 1 Left: Schematic of pretraining-with-memories: some parameters are always used (anchor parameters), others are fetched per input document (memory parameters). **Middle:** Accuracy improvement over baseline when $\simeq 10\%$ of parameters are allocated as memories for a knowledge-intensive task (predicting the atomic numbers of elements), using models with 160M, 410M, and 1.4B parameters, corresponding to rows A2, B2, and C2 in [Table 1](#). **Right:** Elements sorted by their frequency of appearance in the DCLM-Baseline dataset (5 buckets, each with $\simeq 24$ elements). With the proposed *pretraining-with-memories*, we observe significant improvements, especially on long-tail data. While the baseline 1.4B model has only 17% accuracy on the least frequent element bucket, augmenting it with only 10% memory parameters increases the accuracy to 83%.

- 将模型参数剥离为：Anchor Model小模型+存储记忆的参数
- 训练时只对anchor model+选中的记忆参数块梯度更新
- 主要目标是解决常识知识的训练和长尾知识的抗遗忘
- 通过动态加载减少内存的负载

Pretraining with hierarchical memories

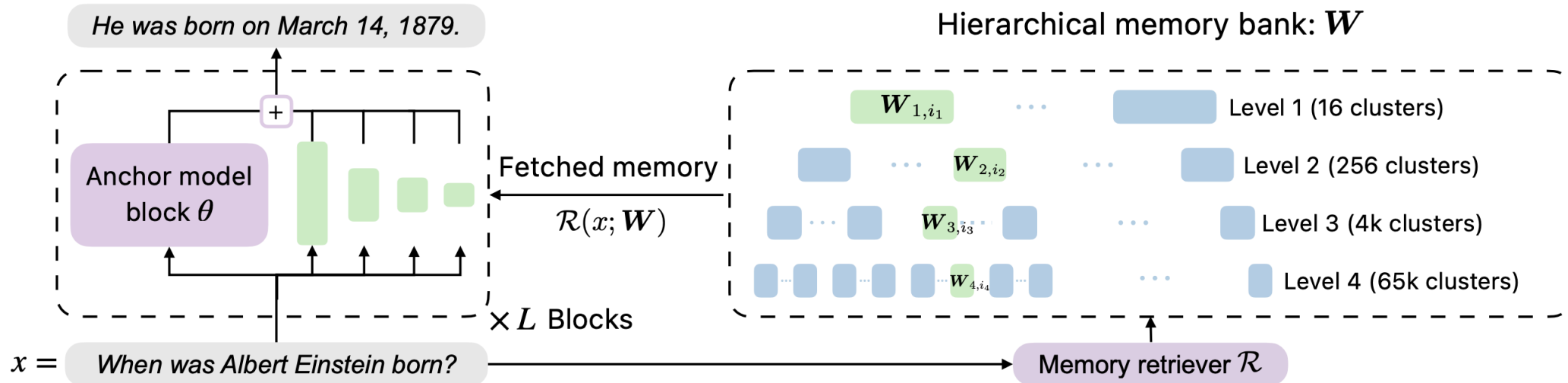


Figure 2 Proposed architecture: For a given context x (such as a question text), the memory retriever module selects relevant parameters from a large set of memory bank parameters. These memory parameters are organized hierarchically based on the hierarchical clustering of the pretraining data. The anchor model, together with the retrieved memories, then responds to the question.

$$\mathcal{L}(x) = - \sum_t \log \mathbb{P}_{\theta, \mathcal{R}(x; \mathbf{W})}(x_t | x_{<t}) \quad |\mathcal{R}(x; \mathbf{W})| \ll |\theta| \ll |\mathbf{W}|$$

Clustering-based memory retriever

- 使用K-means对文本embeddings进行聚类得到 k 簇，并对每个簇进行深层聚类，总共 l 层
- 对于每个输入采用贪心遍历聚类树，取出 l 个记忆参数块

$$\mathcal{R}(x; \mathbf{W}) = [\mathbf{W}_{1,i_1}, \mathbf{W}_{2,i_2}, \mathbf{W}_{3,i_3}, \mathbf{W}_{4,i_4}], \quad \text{where } \mathcal{I}(x) = (i_1, i_2, i_3, i_4).$$

(fetched memory size) is $|\mathcal{R}(x; \mathbf{W})| = s_1 + s_2 + s_3 + s_4$.

Memory Blocks Concatenation

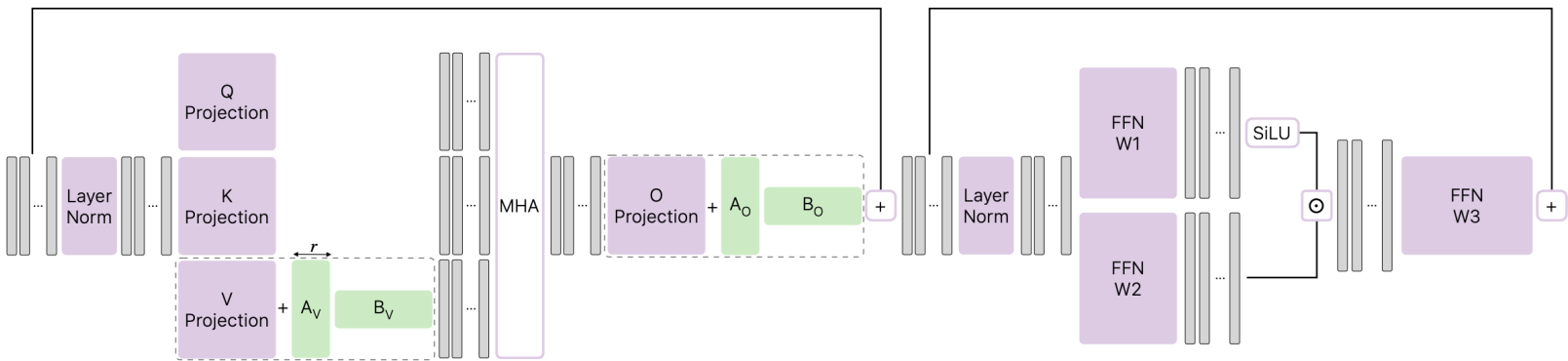


Figure 15 A transformer block with LoRa memories on values and output projection layers.

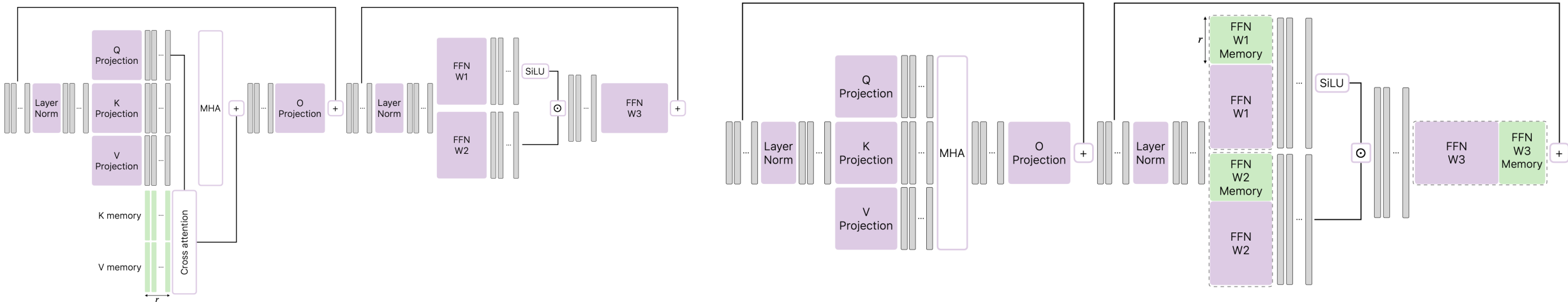
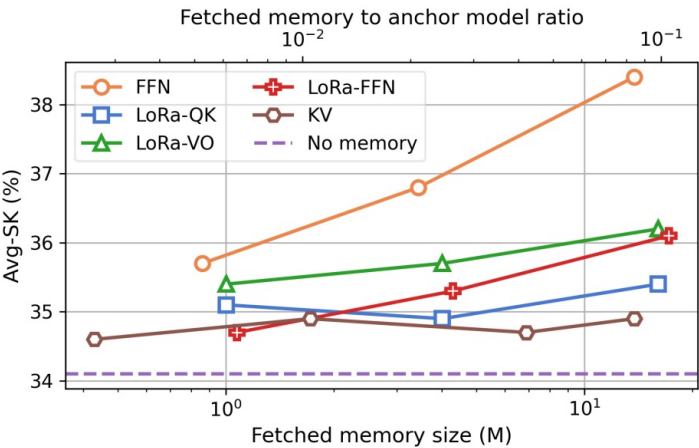


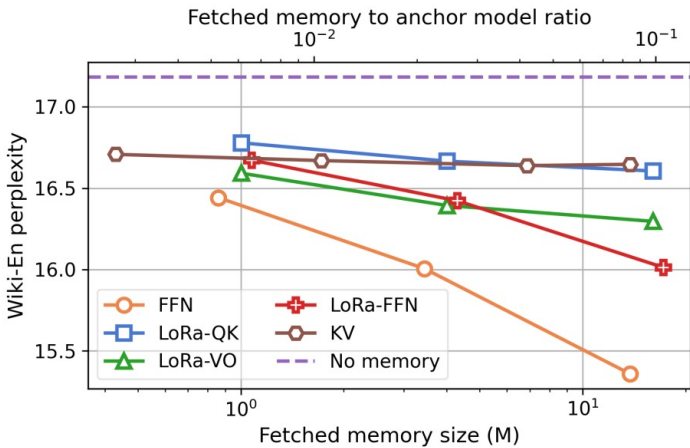
Figure 17 A transformer block with learned KV memories.

Figure 18 A transformer block with FFN memories.

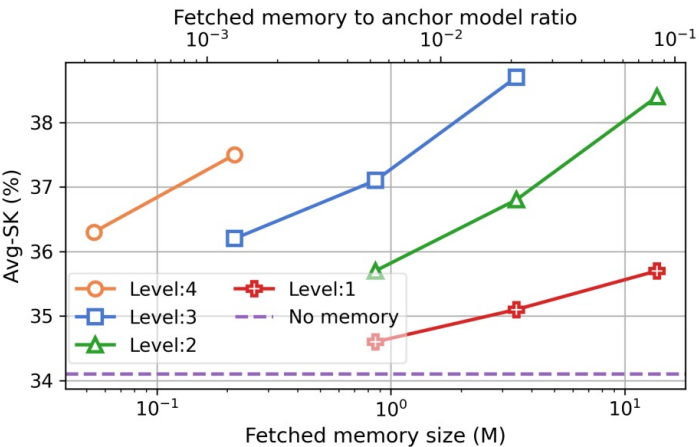
Pretraining with hierarchical memories



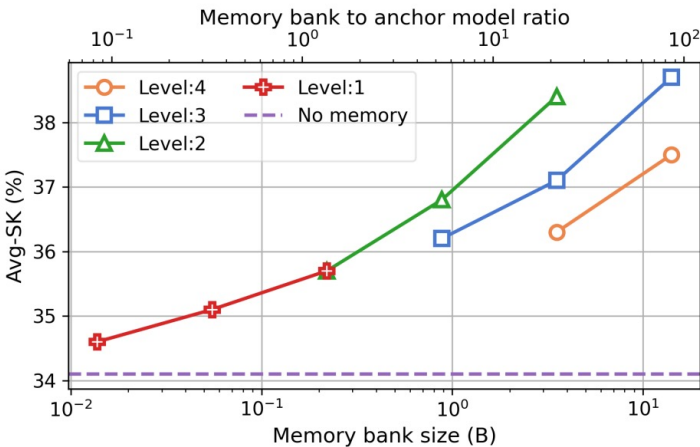
(a)



(b)



(c)



(d)

Figure 3 Effect of **memory type** on Specific-Knowledge benchmarks (a) and Wikipedia perplexity (b). Effect of **memory level** on performance as a function of fetched memory size (c) and bank size (d).

Pretraining with hierarchical memories

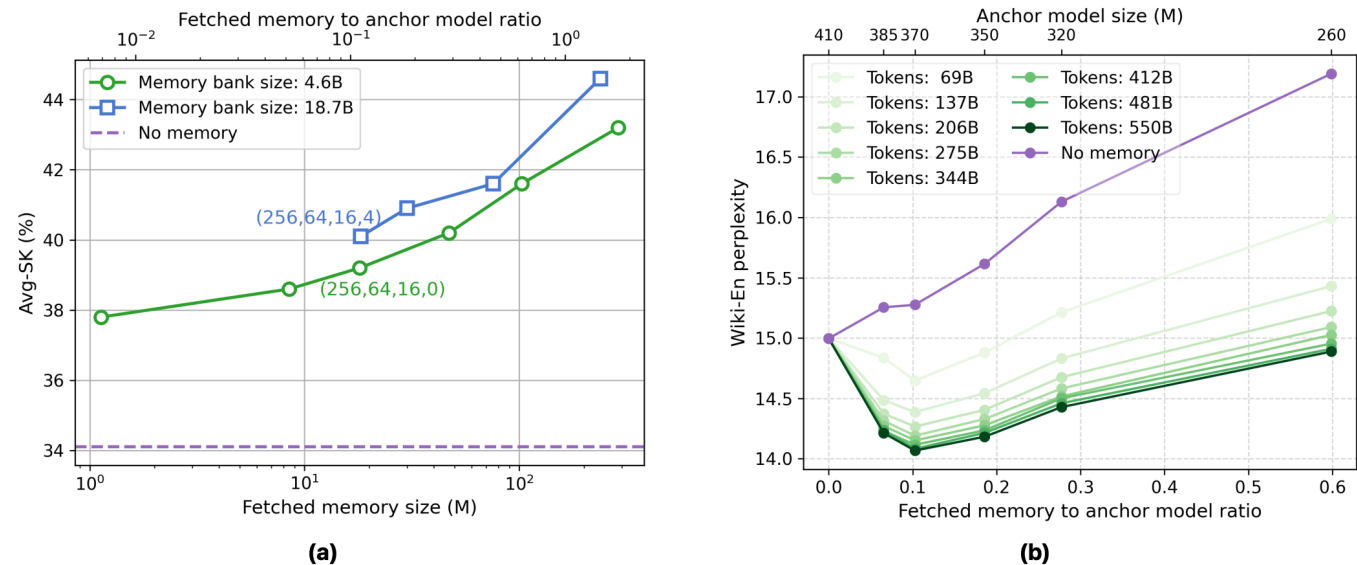


Figure 4 (a) Avg-SK accuracy for different hierarchical memories, demonstrating **performance gain with larger bank size and fetched memory size**. (b) Wiki-En perplexity for different **fetched memory-to-anchor model size ratios**, with the **optimal point at 1:10**. The purple curve shows the perplexity of anchor models without memory. The green curves show the perplexity of models with memory, with different shades of green corresponding to the progress of memory training

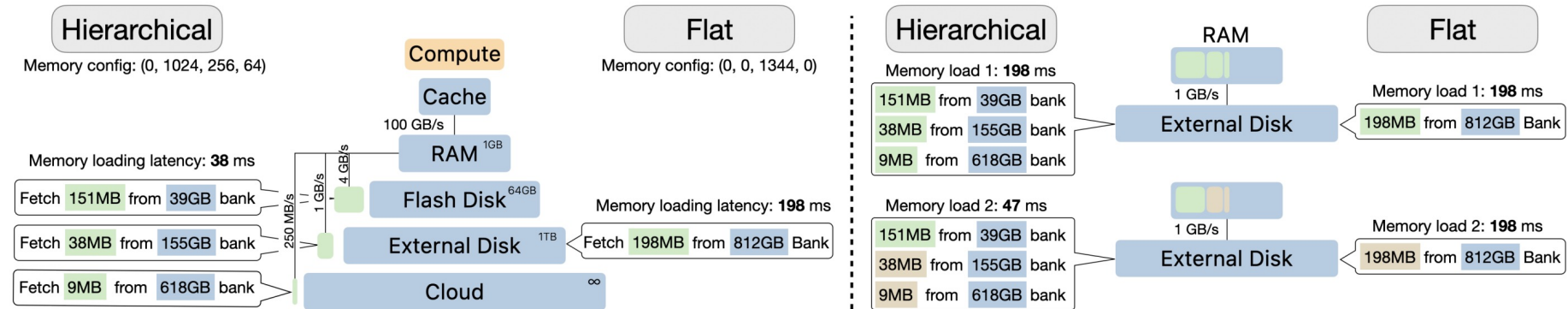
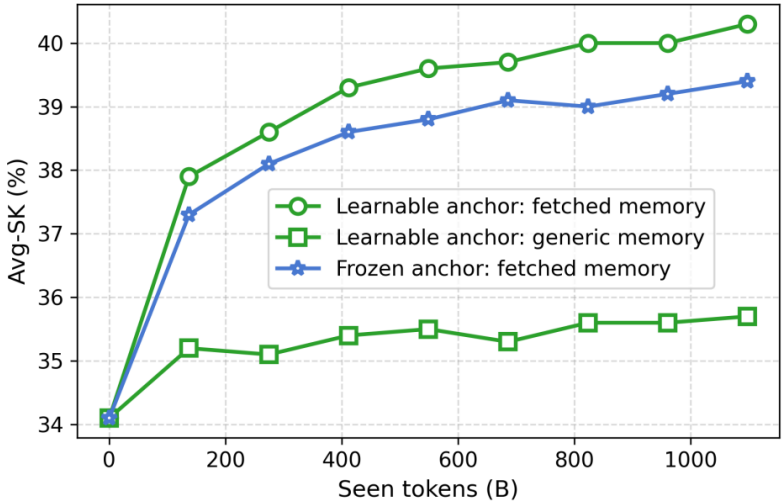


Figure 5 Deployment advantages of hierarchical memories. **Left:** Memory loading latency is reduced by using the hardware hierarchy. **Right:** Latency is reduced by exploiting compositionality over time: larger memories for low-level clusters, once loaded, are less likely to need reloading.

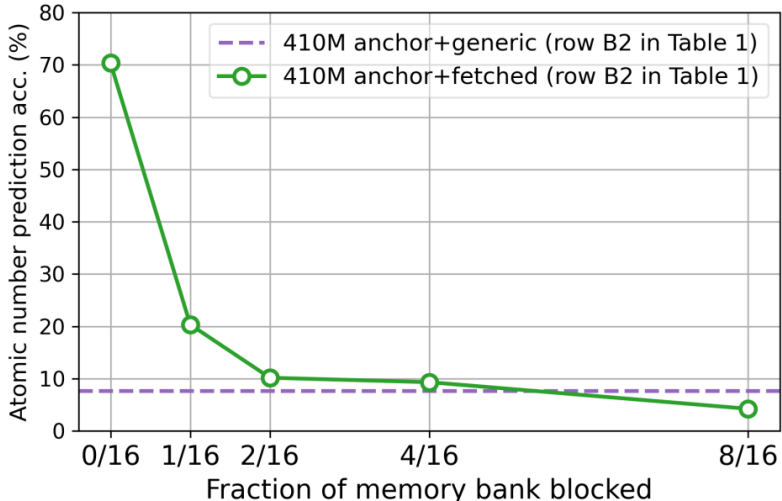
Pretraining with hierarchical memories

Row	Anchor model	Init.	Seen tokens	Cotrain anchor	Memory config	Bank size	Fetch size	Avg-CK (%) ↑		Avg-SK (%) ↑		WikiEn Pplx ↓	
								Generic	Fetches	Generic	Fetches	Generic	Fetches
A1	160M	Scratch	1.1T	n/a	(0,0,0,0)	0	0	45.9		34.1		17.2	
A2		A1	+1.1T	Yes	(256,64,16,0)	4.6B	18M	47.9	48.7	35.7	40.3	16.7	14.2
A3		A1	+1.1T	No				46.6	47.4	34.7	39.2	16.7	15.2
A4		Scratch	2.2T	Yes				46.6	46.7	33.8	39.6	17.8	15.6
B1	410M	Scratch	1.1T	n/a	(0,0,0,0)	0	0	52.3		40.9		13.9	
B2		B1	+1.1T	Yes	(512,128,32,0)	12.7B	50M	55.5	56.1	41.8	45.9	13.8	12.4
B3		Scratch	2.2T	n/a	(0,0,0,0)	0	0	53.2		41.1		13.8	
C1	1.4B	Scratch	1.1T	n/a	(0,0,0,0)	0	0	61.2		49.7		10.8	
C2		C1	+1.1T	Yes	(768,256,16,0)	21.1B	153M	64.4	64.5	51.3	54.9	11.0	10.2

Table 1 Summary of results for pretraining with memories at different parameter scales.



(a)



(b)

Figure 6 (a) Performance improvements on Avg-SK when co-training memories and anchor model parameters jointly during training. (b) Effect of blocking parts of the memory bank from retrieval.

Pretraining with hierarchical memories

Cluster config.	Embed. model	Memory config.	AVG-CK (%)	AVG-SK (%)
$p = 4$ $k = 16$	MiniLM V6	(64, 16, 4, 1)	42.8	36.6
$p = 4$ $k = 16$	GTE small	(64, 16, 4, 1)	42.6	36.2
$p = 2$ $k = 256$	MiniLM V6	(84, 1)	43.1	37.5

Table 2 Clustering ablation.

Pretrained model	Bank size	Fetch size	Avg-CK (%) $\uparrow$	Avg-SK (%) $\uparrow$	Atomic Number Acc. (%) $\uparrow$
Gemma 3 270M	0	0	44.2	34.3	1.7
+ memory	5.9B	23.2M	44.8	38.2	49.2
Qwen 2.5 0.5B	0	0	53.9	40.6	53.4
+ memory	11.1B	43.4M	52.1	44.5	90.4
Llama 3.2 1B	0	0	58.9	46.6	96.6
+ memory	14.1B	102.2M	57.6	50.5	96.6

Table 4 Learning memory on top of pretrained open-weight models post-hoc. All trainings use 1.1 trillion tokens from DCLM.

Num tokens	Model setup	Bank Loc.	Routing time	Fetch time	Prefill and Gen. time	Total time
20	Baseline	n/a	n/a	n/a	507 ms	507 ms
20	w/ mem.	HBM	7 ms	0	534 ms	541 ms
20	w/ mem.	DRAM	7 ms	23 ms	534 ms	564 ms
40	Baseline	n/a	n/a	n/a	1017 ms	1017 ms
40	w/ mem.	HBM	7 ms	0	1040 ms	1047 ms
40	w/ mem.	DRAM	7 ms	23 ms	1040 ms	1070 ms

Table 3 Runtime cost comparison on a single H100 GPU.

Anchor model	Inference setup	Bank size	FLOPs	Avg-CK 0-shot	Avg-SK 0-shot
160M	Baseline	0	$\times 1$	43.6	32.8
	RAG-DCLM	70 TB	$\times 2.3$	42.4	32.6
	RAG-Wiki	21 GB	$\times 1.7$	42.4	35.0
	10% Memory	9 GB	$\times 1.11$	45.3	38.4
410M	Baseline	0	$\times 1$	48.6	38.5
	RAG-DCLM	70 TB	$\times 2.3$	48.2	38.1
	RAG-Wiki	21 GB	$\times 1.7$	47.3	41.6
	10% Memory	25 GB	$\times 1.1$	52.0	44.5
1.4B	Baseline	0	$\times 1$	56.0	46.9
	RAG-DCLM	70 TB	$\times 2.3$	55.8	46.1
	RAG-Wiki	21 GB	$\times 1.7$	55.5	49.2
	10% Memory	42 GB	$\times 1.1$	59.3	52.4

Table 5 Comparison with vanilla RAG.